

Theory:

- 1) The basic structure is `for (initialization, condition, increment)`
 - * Initialization runs once at the beginning.
 - * Condition is checked before every iteration.
 - * Increment updates the loop variable after each iteration.
- 2) A while loop checks the condition before executing the loop body, so it may execute zero times. A do-while loop executes the body first and checks the condition afterward so it always executes at least once.
- 3) A do-while loop executes its body before checking the condition. Therefore, even when the condition is false initially, the body runs once.
- 4) `for...in` is mainly used to iterate over the enumerable property keys of an object. It returns the property names/keys rather than their values.
- 5) `for...of` is used to iterate over the values of an iterable, such as an array, string, Map, or set.
 - * `for...in` $\rightarrow$ iterates over keys/indexes
 - * `for...of` $\rightarrow$ iterates over values.
- 6) Because `for...in` returns the array's indexes as strings, not the actual values, it is intended primarily for object properties. `for...of` is generally more suitable for iterating through array values.

7) A nested loop is loop placed inside another loop. It is commonly used for processing grids, matrices, multiplication tables or combinations of elements from two collections.

8) An infinite loop occurs when the loop's termination condition never becomes false or when the loop variable is not updated correctly. It can be safely stopped using a break statement when a required condition is reached.

9) In a nested loop, the inner loop executes multiple times for each iteration of the outer loop. If both loops depend on the size of the data, the number of operations can grow approximately as $O(n^2)$, making it slower for large datasets.

10) break terminates the loop completely.

Continue skips the current iteration and proceeds to the next iteration. They can be used in both while and for loops.

Output prediction:

11) 1
2
3

12) 5
4
3
2
1

13) 10

14) a
b

15) 10
20
30

16) $\begin{matrix} h \\ i \end{matrix}$

17) $\begin{matrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \end{matrix}$

18) 3

19) string
string
string

20) 15

Error Findings:

24) This creates an infinite loop because true is always true and there is no break or other termination condition. It can cause the program/browser to hang.

21) This creates an infinite loop because i starts at 0 and is decreased using $i--$. Therefore, it becomes -1, -2, -3... and the condition $i \leq 5$ remains true.

22) This creates an infinite loop because true is always true and there is no break or other termination condition. It can cause the program/browser to hang.

23) O/P:

0
1
2

because for...in iterates over the indexes, not the values

24) This is not an error. It prints:

True

once because do-while executes the body before checking

the condition.

25) This causes a `Type Error` because `undefined` does not have a `length` property.

26) If the same mutable counter variable is reused by both loops, the inner loop can modify the outer loop's counter. This causes incorrect loop control and may result in unexpected output or an infinite loop.

27) `for...of` works only with iterable objects. A plain JavaScript object `{}` is not iterable by default, so using `for...of` directly on it causes a `Type Error`. For a plain object, `for...in` or `object.keys()`, `object.values()`, or `object.entries()` can be used.

28) `break` terminates only the nearest enclosing loop in which it appears. Therefore, when `break` is used inside an inner nested loop, it exits the inner loop only. The outer loop continues its execution.

29) Array indexes range from 0 to `arr.length - 1`. Using `i < arr.length` allows `i` to become `arr.length`, which is one position beyond the last valid index. Therefore, `arr[i]` returns `undefined` at that final iteration.

30) When the array is modified ~~even~~ during `for...of` iteration its contents or length can change while the loop is still running. This can change which elements are visited and how many iterations occur, resulting in unexpected behavior. Therefore, modifying an array's length while iterating over

it should generally be avoided unless it is intentional

Coding:

```
31) for (let i = 1; i <= 10; i++) {  
    console.log('7 x ' + i + ' = ' + (7 * i));  
}
```

O/P

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

```
32) let i = 10;  
    while (i >= 1) {  
        console.log(i);  
        i--;  
    }
```

O/P

10
9
8
7
6
5
4
3
2
1

```
33) let i = 1;  
    do {  
        console.log(i);  
        i++;  
    } while (i <= 5);
```

O/P

1
2
3
4
5

The do-while body executes at least once before the condition is checked.

```
34) const person = {  
    name: "John",  
    age: 25,  
    city: "Chennai",  
};
```

```
for (let key in person) {  
    console.log(key, person[key]);  
}
```

O/P

name John
age 25
city Chennai

```
35) const numbers = [10, 20, 30, 40, 50];  
    let sum = 0;
```

```

for (let number of numbers) {
    Sum += number;
}
Console.log(Sum);

```

O/p: 150

```

36) for (let i=0; i<3; i++) {
    let row = '';
    for (let j=0; j<3; j++) {
        row += '*';
    }
    Console.log(row);
}

```

O/p

```

* * *
* * *
* ( * )

```

```

37) const numbers = [1, 3, 7, 9, 8, 10, 12];

```

```

for (let number of numbers) {
    if (number % 2 === 0) {
        Console.log(number);
        break;
    }
}

```

O/p: 8

// break stop the loop immediately after finding the first even number

```

38) const str = "javascript";

```

```

for (let ch of str) {
    if ('aeiou'.includes(ch)) {
        continue;
    }
    Console.log(ch);
}

```

O/p

```

j
a
v
a
s
c
r
i
p
t

```

// Here, continue skips the vowels and moves to the next character.

```

39) const arr1 = [1, 2, 3, 4];

```

```

const arr2 = [2, 3, 4, 5];

```

```

const target = 6;

```

```
for (let i = 0; i < arr1.length; i++) {
```

```
  for (let j = 0; j < arr2.length; j++) {
```

```
    if (arr1[i] + arr2[j] === target) {
```

```
      console.log(arr1[i], arr2[j]);
```

```
    }  
  }  
}
```

40) let count = 0;

```
for(;;) {
```

```
  count++;
```

```
  if (count === 5) {
```

```
    break;
```

```
  }
```

```
  console.log(count);
```

Real World scenario:

41) for...of fits naturally because it directly iterates over each item in the shopping cart. It allows each item's price and quantity to be processed easily.

42) A while loop fits this pattern because the number of iterations is not known in advance. The loop can continue while the job status is not done.

43) A nested loop is natural because one loop can handle the weeks and the inner loop can handle the days within each week.

O/P: 5

for(;;) creates a loop without initialization condition, or increment sections. The break statement provides the exit condition.

44) `for...in` is the appropriate choice because it iterates over the enumerable property keys of an object. The keys can then be used to access the corresponding values.

45) `for...of` with `break` is an appropriate combination. `for...of` iterates over the user objects, and `break` stops the loop immediately when the matching ID is found.

46) A `for` loop is suitable because the maximum number of attempts is known: 5 attempts. The `break` statement can stop the loop early when the API call succeeds.

47) `for...of` directly returns the value of the array, while `for...in` returns the indexes/property keys. Therefore, `for...of` is generally clearer and more appropriate when processing array elements.

48) Always double-check that the loop has a proper termination or exit condition and that the loop-control variable or condition is updated correctly. An infinite synchronous loop can cause the browser tab to freeze.

49) A nested loop is needed because each product must be compared with other products. With large datasets, nested loops can require many operations and may have approximately $O(n^2)$ time complexity, so performance

Should be considered.

50) `for...` of `handles Map` and `set` clearly because both are iterable, `for...` is designed for an enumerable property keys of objects and is the appropriate way to iterate through the contents of a `Map` or `set`.